

Parallel programming

We were asked to do 5 problems in C and in Elixir using parallel programming and then compare the time it takes from sequential to parallel.

1.-

1. The **factorial** of a positive integer n is defined as:

$$n! = \prod_{i=1}^n i = 1 \times 2 \times 3 \times \cdots \times (n-2) \times (n-1) \times n$$

Write a function that takes n as input and returns the number of bits equal to 1 found in the binary representation of $n!$. For example, if $n = 6$ the return value should be 4, since $6! = 720$ and $720_d = 1011010000_b$, and there are 4 bits equal to 1.

In C

For a sequential result

```
berges@LAPTOP-KHI14BAS:/mnt/c/Users/berge/OneDrive/SEMESTRE 4/elixir/concurrency/finished$ time ./factorial 19
Launching threads
Created thread: 0 [1, 19]
Going to pthread_join
Running thread 0
Number of 1's in binary of 19! = 17

real    0m0.007s
user    0m0.002s
sys     0m0.001s
berges@LAPTOP-KHI14BAS:/mnt/c/Users/berge/OneDrive/SEMESTRE 4/elixir/concurrency/finished$
```

For parallel

```
berges@LAPTOP-KHI14BAS:/mnt/c/Users/berge/OneDrive/SEMESTRE 4/elixir/concurrency/finished$ time ./factorial 19 8
Launching threads
Created thread: 0 [1, 2]
Running thread 0
Created thread: 1 [3, 4]
Running thread 1
Created thread: 2 [5, 6]
Running thread 2
Created thread: 3 [7, 8]
Running thread 3
Created thread: 4 [9, 10]
Running thread 4
Created thread: 5 [11, 12]
Running thread 5
Running thread 6
Created thread: 6 [13, 14]
Created thread: 7 [15, 19]
Going to pthread_join
Running thread 7
Number of 1's in binary of 19! = 17

real    0m0.008s
user    0m0.000s
sys     0m0.004s
berges@LAPTOP-KHI14BAS:/mnt/c/Users/berge/OneDrive/SEMESTRE 4/elixir/concurrency/finished$
```

There's not much of a difference between the times as the factorial problem does not require much computing power. The result is correct because

In elixir

In elixir i didn't made the binary and 1's counter because it wasn't required
Sequential

```
iex(6)> :timer.tc(fn -> ParallelFactorial.factorial_parallel(19,1) end)
[{1, 19}]
{204, 121645100408832000}
```

Parallel

```
iex(4)> :timer.tc(fn -> ParallelFactorial.factorial_parallel(19,8) end)
[{1, 3}, {4, 6}, {7, 9}, {10, 12}, {13, 15}, {16, 18}, {19, 19}]
{8908, 121645100408832000}
```

2.-

2. The **sum of the prime numbers** smaller than 10 is:

$$2 + 3 + 5 + 7 = 17$$

Write a function that takes a positive integer n and computes the sum of all prime numbers smaller than or equal to n . To ensure the process stresses the CPU, use the following algorithm to identify prime numbers.

Create a function to **identify prime numbers**, that takes a number x as input, and returns *true* or *false*

1

- If x is less than 2, return *false*.
- If x is 2, return *true*.
- For any number i from 2 to $\lceil \sqrt{x} \rceil$:
 - Return *false* if x is divisible by i .
 - Otherwise, repeat with $i + 1$.
- Return *true* when i becomes larger than $\lceil \sqrt{x} \rceil$.

In C

Sequential

```
berges@LAPTOP-KHI148AS:/mnt/c/Users/berge/OneDrive/SEMESTRE 4/elixir/concurrency/finished$ time ./primes 1000
Launching threads
Created thread: 0
Going to pthread_join
Running thread 0
The sum of primes up to 1000 is 76127

real    0m0.008s
user    0m0.000s
sys     0m0.003s
```

Parallel

```
Running thread 96
Created thread: 97
Running thread 97
Created thread: 98
Running thread 98
Running thread 99
Created thread: 99
Going to pthread_join
The sum of primes up to 1000 is 76127

real    0m0.027s
user    0m0.003s
sys     0m0.024s
```

same result but time again not so useful with small numbers

In elixir

sequential

```
iex(2)> :timer.tc(fn -> Primes.prime_parallel(1000, 1)end)
[{1, 1000}]
{819, 76127}
```

Parallel

```
{281, 290},
{291, 300},
{301, 310},
{311, 320},
{321, 330},
{...},
...
]
{1228, 76127}
iex(4)>
```

again 100 threads

3.-

3. **Numerical integration** is a method to compute an approximation of the area under the curve of a function, especially when the integral cannot be solved exactly. For example, the value of the constant π can be defined by this integral:

$$\pi = \int_0^1 \frac{4}{1+x^2} dx$$

Instead of finding the exact integral, we can approximate the solution using numerical integration. The following C language code is an implementation of the Rectangle method to solve the integral previously shown. The approximation of the area under the curve is obtained by defining a number n of rectangles, finding their horizontal medium point `mid`, and their `height`, which is the evaluation of the function at the medium point. The heights of all rectangles are summed and multiplied by the `width` of the rectangles, to determine the approximation of the area and the value of π .

```
double compute_pi(long n) {
    double sum = 0.0;
    double width = 1.0 / n;
    for (long i = 0; i < n; i++) {
        double mid = (i + 0.5) * width;
        double height = 4.0 / (1.0 + mid * mid);
        sum += height;
    }
    return width * sum;
}
```

Write an equivalent function in the language indicated. In general, a better approximation of π is obtained for larger values of n . Compare your results against the first 20 decimals in π :

3.14159265358979323846...

Bear in mind that floating point numbers implemented with 64-bit doubles have a precision of up to 16 decimal places, but further rounding errors will be accumulated after several operations.

In C

sequential

```
berges@LAPTOP-KHI148AS:/mnt/c/Users/berge/OneDrive/SEMESTRE 4/elixir/concurrency/finished$ time ./num_integration 100
Launching threads
Created thread: 0
Going to pthread_join
Running thread 0
The approximation of pi is 3.141600986923125

real    0m0.006s
user    0m0.002s
sys     0m0.000s
berges@LAPTOP-KHI148AS:/mnt/c/Users/berge/OneDrive/SEMESTRE 4/elixir/concurrency/finished$
```

Parallel

```
Created thread 7
Running thread 7
Created thread: 8
Running thread 8
Created thread: 9
Going to pthread_join
Running thread 9
The approximation of pi is 3.141600986923125

real    0m0.008s
user    0m0.001s
sys     0m0.005s
berges@LAPTOP-KHI148AS:/mnt/c/Users/berge/OneDrive/SEMESTRE 4/elixir/concurrency/finished$
```

with 10 threads

In Elixir

sequential

```
iex(3)> :timer.tc(fn -> Num_integration.numint_parallel(100, 1)end)
[{0, 99}]
{204, 3.1416009869231254}
```

Parallel

```
berge@LAPTOP-KHI14BAS MINGW64 ~/OneDrive/SEMESTRE 4/Elixir/Concurrency/finished
$ iex num_integration.exs
[
  {0, 9},
  {10, 19},
  {20, 29},
  {30, 39},
  {40, 49},
  {50, 59},
  {60, 69},
  {70, 79},
  {80, 89},
  {90, 99}
]
{512, 3.1416009869231245}
```